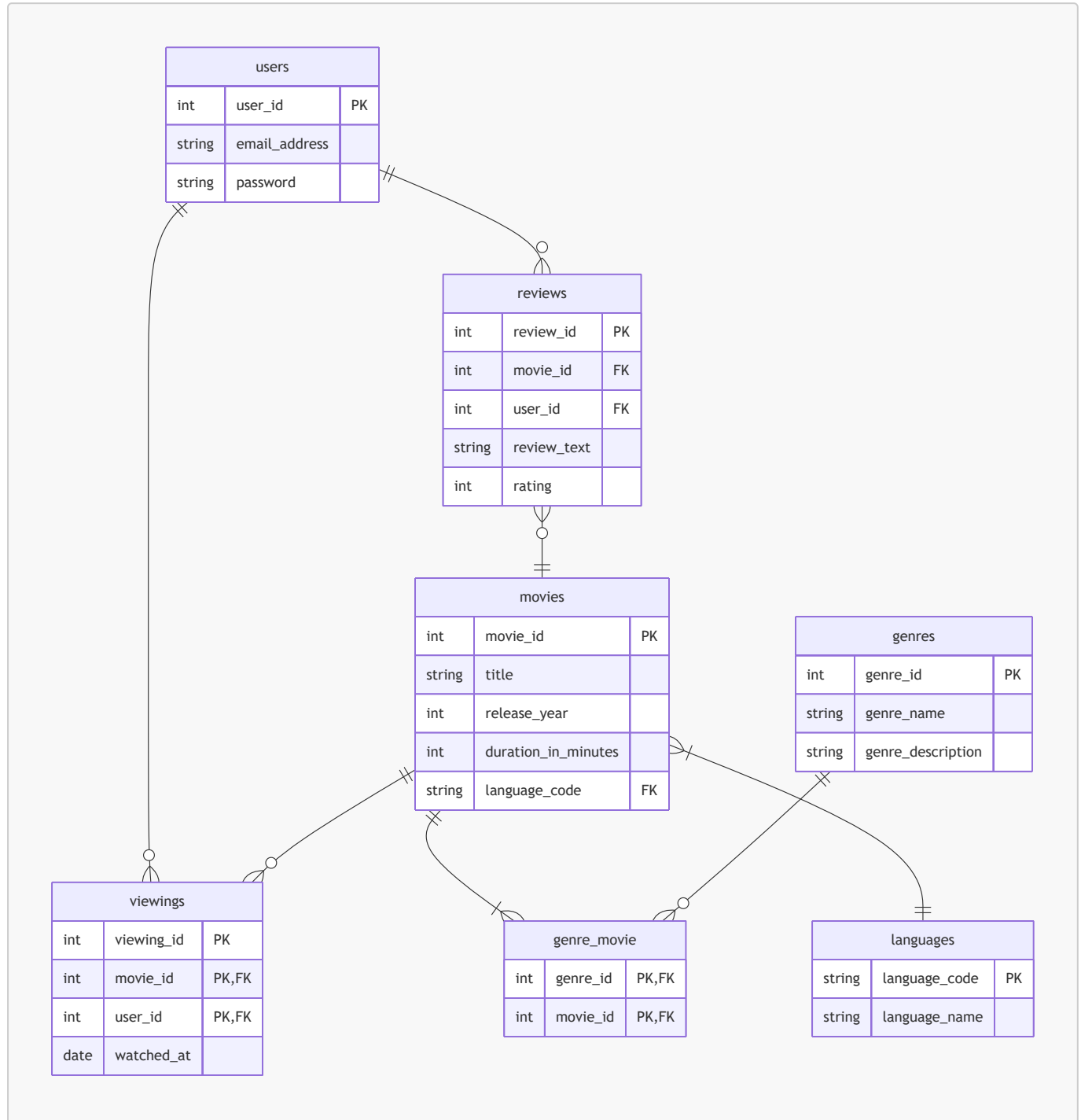


Joins and Subqueries

- AND in the JOIN becomes important for LEFT JOINs so introduce it here.

Have a look at the following ERD written in mermaidjs.



- Run the subqueries first to see what they do.

For an introductory course, you should teach CTEs as a modern replacement for subqueries inside the FROM and JOIN clauses. Keep Scalar Subqueries for the WHERE clause (like WHERE duration < (SELECT AVG...)), but tell your students that anytime they feel tempted to nest a large query inside a JOIN, they should build a CTE instead.

Would you like to see how to rewrite your highest-rated movie genre query into a clean CTE format for your lesson notes?

Nearly all subqueries can be written as joins

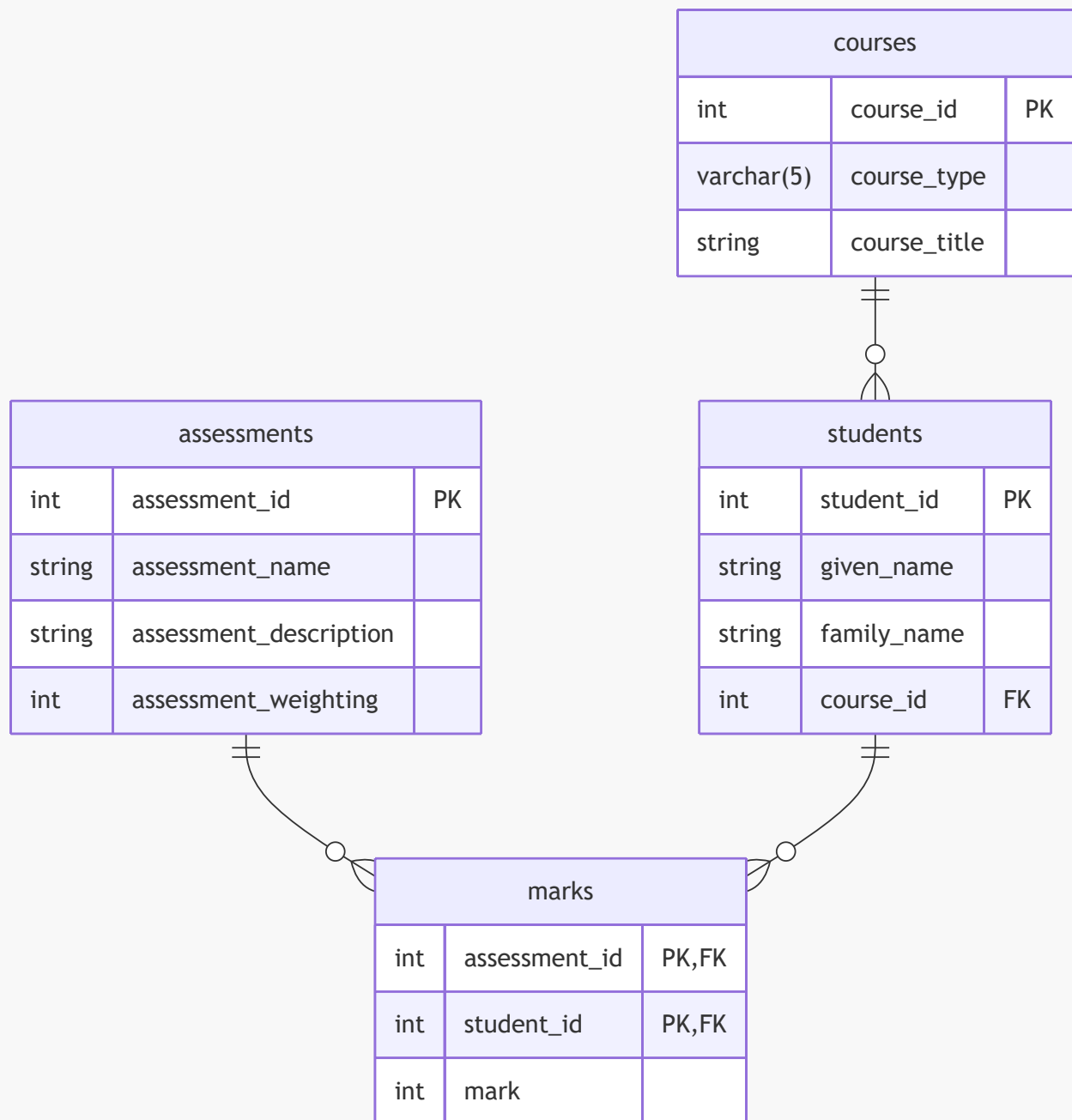
Behind the scenes the database engine will often run a join instead.

Use a JOIN when you want to glue columns from two tables together to see them side-by-side on your screen.

Use an IN subquery when you only want columns from one table, but you need to filter it using a list from another table.

It may be worth doing IN vs JOIN for short-circuit vs exhaustive match.

Student Assessment Example



Left Join

The LEFT JOIN Use Case: Finding Inactive Records.

A LEFT JOIN keeps every single row from the left table, even if it cannot find a match in the right table. When a match is missing, the database pads the columns on the right with NULL.

The Business Scenario: The marketing team wants to find a list of all registered users who have never watched a single movie so they can send them a "Welcome back, here is a discount" email.

```
SELECT u.email_address, v.watched_at
FROM users u
```

```
LEFT JOIN viewings v ON u.user_id = v.user_id
WHERE v.viewing_id IS NULL;
```

Right Join

The RIGHT JOIN Use Case: Checking Catalog Coverage

A RIGHT JOIN does the exact opposite: it keeps every single row from the right table, regardless of matches on the left.

The Business Scenario: The content curation team wants to find unpopular genres in the catalog. Specifically, they want a list of all genres that currently have zero movies assigned to them.

```
SELECT g.genre_name, gm.movie_id
FROM genre_movie gm
RIGHT JOIN genres g ON gm.genre_id = g.genre_id
WHERE gm.movie_id IS NULL;
```

This could also be written as a left join.

Example subqueries

Simple Scalar Subquery (Returns 1 Value)

Find all movies that have a duration longer than the average movie length.

```
SELECT title, duration_in_minutes
FROM movies
WHERE duration_in_minutes > (
    SELECT AVG(duration_in_minutes)
    FROM movies
);
```

Simple List Subquery (Using IN)

Find the titles of all movies that have received a 5-star rating.

```
SELECT title
FROM movies
WHERE movie_id IN (
    SELECT movie_id
    FROM reviews
    WHERE rating = 5
);
```

This can also be written as a join

```
SELECT DISTINCT m.title
FROM movies m
INNER JOIN reviews r ON m.movie_id = r.movie_id
WHERE r.rating = 5;
```

The Exclusion Subquery (Using NOT IN)

Find the email addresses of all users who have never watched a single movie.

```
SELECT email_address
FROM users
WHERE user_id NOT IN (
    SELECT DISTINCT user_id
    FROM viewings
);
```

Aggregate Subquery

Find all movies that were released in the same year as the longest movie in the database.

```
SELECT title, release_year
FROM movies
WHERE release_year = (
    SELECT release_year
    FROM movies
    ORDER BY duration_in_minutes DESC
    LIMIT 1
);
```

- Explain Analyse

Slide 1: Introduction to Query Execution

Slide Bullets

- :The SQL Illusion: SQL tells the database what data you want, not how to get it. -The Query Optimizer: A built-in calculator that parses your code and creates an Execution Plan. -The Goal: Minimize "Disk I/O" (reading data from hard drives) and memory usage. -The Question: Does a subquery run slower than a join?

Students often think that writing a query differently changes how hard the computer has to work. In modern databases, that's not always true. Today we are going to look behind the curtain at the Query Optimizer. This is the brain of the database that transforms your SQL syntax into raw operational steps. We'll see why the

database engine sometimes treats subqueries and joins exactly the same, and when one completely outperforms the other.

Slide 2: The Myth of the "Slower" Subquery

- The Old Rule: "Subqueries are slow because they run the inner loop over and over." (Outdated!)
- Subquery Flattening: Modern optimization engines automatically convert simple subqueries into JOIN structures internally.
- Example (The 5-Star Movie Filter): WHERE movie_id IN (SELECT movie_id FROM reviews...) Postgres automatically unrolls this into an internal hash join or semi-join.

If you read older database textbooks, they will tell you to avoid subqueries because they are slow. That is a myth in 2026. Modern database engines use a technique called 'subquery flattening' or 'pull-up'. When you write an IN subquery, the optimizer looks at it, realizes it's mathematically identical to a join, and rewrites it under the hood before running it. The execution speed for our 5-star movie query will be identical in both styles.

Slide 3: Where Subqueries Lose (Correlated Subqueries)

- Non-Correlated Subquery: Runs exactly once, feeds the result to the outer query.
- Correlated Subquery: References columns from the outer query.
- The Execution Danger: The inner query must execute once for every single row processed by the outer query. Performance Scaling: Generates an $O(N^2)$ "Nested Loop" complexity penalty.

Where subqueries absolutely can destroy your server's performance is when they are correlated. If the inner query relies on data from the outer query—like checking a user's specific viewing history row-by-row—the engine can no longer run the inner query just once. If your outer table has 10,000 movies, the inner query runs 10,000 times. If you can rewrite a correlated subquery into a standard INNER JOIN, you will see a massive speed increase.

Slide 4: Where Joins Lose (The Duplication & Memory Trap)

- Join Bloat: Joins physically combine rows, creating massive temporary data structures in memory.
- The Aggregate Problem: Joining movies to reviews multiplies movie rows by the number of reviews they have. -Hidden Costs: Forcing the database to use DISTINCT or GROUP BY to clean up a join requires an expensive secondary memory-sorting operation.

Teacher's Talking Script: "Joins aren't always perfect either. When you join our movies table to a high-volume reviews table, the database physically creates a massive combined dataset in temporary memory. If a movie has 5,000 reviews, that movie's title is duplicated 5,000 times in RAM. If you only care if a movie has a review, an EXISTS subquery is much faster because it stops searching the moment it finds the first match, saving memory."

Slide 5: The "NOT IN" vs. "LEFT JOIN" Danger Zone

- The Goal: Find users with zero viewings.
- The Trap: WHERE user_id NOT IN (SELECT user_id FROM viewings)
- The Flaw: If a single row in viewings contains a NULL value, a NOT IN query returns zero results globally due to Three-Valued Logic (AND NULL evaluates to unknown).

- The Fix: Use LEFT JOIN ... WHERE viewings.user_id IS NULL. It is faster and completely immune to the NULL trap.

Teacher's Talking Script:"Here is a critical warning for your practical lab assignments: never use NOT IN if the underlying column allows missing data. If your subquery list contains even a single NULL value, SQL's logic engine breaks and the entire query will return empty results. To find missing data cleanly and safely, a LEFT JOIN combined with an IS NULL check is the industry-standard execution pattern."

Slide 6: Summary — How to Choose in Production

- Use Subqueries for readability when dealing with isolated calculations (averages, maximums).
- Use Subqueries (EXISTS) when you just need to check for a match without pulling data from the second table.
- Use Joins when you actually need to display columns from both tables in your final SELECT output.
- Always use EXPLAIN ANALYZE in your terminal to see the true cost!

Teacher's Talking Script:"To summarize: use subqueries when you want clean code for calculations, or when you want to filter a table based on an existential check. Use joins when you actually want to display information from both tables on the screen together. And when in doubt, run the EXPLAIN ANALYZE command in your terminal. This forces the database to show you exactly how many milliseconds each path took, ending any performance debate instantly."